

IFMA — Campus Coelho Neto

Tecnologia em Análise e Desenvolvimento de Sistemas

Padrões de Projetos e Teste de Software • 6º Período — 2026.1

APOSTILA

Testes de Software

*Fundamentos, Técnicas, Automação e
Qualidade*

Prof. Dr. Diogo Vinícius de Sousa Silva, CTFL

Material de Apoio

Instituto Federal do Maranhão • Abril/2026

Sumário

Apresentação	
	3 Capítulo 1 — Fundamentos de Teste de
Software.....	4
testar?.....	4 1.1 Por que
Defeito e Falha.....	4 1.2 Erro,
Verificação × Validação	5 1.3
Sete Princípios de Teste (ISTQB)	5 1.4 Os
ISTQB e a certificação CTFL v4.0.1.....	6 1.5 A
Processo de Teste e Níveis de Teste.....	6 1.6
Tipos de Teste	7 1.7

Capítulo 2 — Técnicas de Teste	9
2.1 Testes estáticos e revisões	9
2.2 Técnicas de caixa-preta	
2.3 Técnicas de caixa-branca	
2.4 Técnicas baseadas em experiência	11
2.5 Teste de mutação	11
2.6 Qual técnica aplicar em cada contexto	12
Capítulo 3 — Automação, Ferramentas e Qualidade	13
3.1 Pirâmide de testes e ROI da automação	13
3.2 JUnit 5 e JUnit 6	13
3.3 Mockito e Test Doubles	15
3.4 TDD e BDD	16
3.5 JaCoCo e cobertura de código	16
3.6 SonarQube 2025 e Clean as You Code	
3.7 CI/CD, Quality Gates e métricas DORA	17
3.8 Panorama 2026 de ferramentas	18

Referências	19
--------------------	-----------

Apresentação

Esta apostila foi elaborada como material de apoio para as aulas da disciplina Padrões de Projetos e Teste de Software. O texto complementa os slides de aula, aprofunda conceitos, traz exemplos de código e, sempre que possível, conecta cada tópico à certificação internacional ISTQB® Certified Tester Foundation Level (CTFL) v4.0.1 — referência global de vocabulário em teste de software.

Ao longo do material você encontrará três categorias de elementos didáticos: (a) definições objetivas, (b) exemplos práticos com trechos de código e diagramas e (c) pílulas "CTFL" destacando pontos cobrados no exame de certificação, úteis tanto para o ENADE quanto para quem pretende futuramente fazer a prova.

Boa leitura e bons testes!

Prof. Dr. Diogo Vinícius de Sousa Silva, CTFL

Capítulo 1 — Fundamentos de Teste de Software

1.1 Por que testar?

Pode parecer óbvio, mas vale lembrar: software não é como um produto físico que passa por um controle de qualidade no fim da linha de produção. Cada execução pode revelar um comportamento novo, dependente de estado, dados e ambiente. Sem testar de forma estruturada, entregar software funcional é uma questão de sorte.

Três casos clássicos dimensionam o impacto financeiro e humano de falhas de software:

- Ariane 5 (1996) — um erro de conversão de 64 para 16 bits levou o foguete a explodir 37 segundos após a decolagem. Prejuízo estimado: US\$ 370 milhões.
- Therac-25 (1985-1987) — condição de corrida no controle do acelerador linear causou sobredose de radiação em seis pacientes, com morte confirmada em três.
- Knight Capital (2012) — um deploy parcial reativou código morto, e a empresa perdeu US\$ 440 milhões em 45 minutos, indo à falência técnica.

Estudos clássicos do IBM Systems Sciences Institute mostram que o custo de corrigir um defeito em produção pode ser até 100 vezes maior do que o custo de corrigi-lo durante a fase de requisitos. Este é um dos motivos pelos quais o CTFL consagra o princípio do teste antecipado (shift-left), que discutiremos em 1.4.

PÍLULA CTFL 1.1

O CTFL v4.0 cobra a distinção entre teste (atividade dinâmica ou estática) e debugging (atividade do desenvolvedor para localizar e corrigir defeitos já reportados).

1.2 Erro, Defeito e Falha

O vocabulário ISTQB, consolidado no ISTQB Glossary v4.5 (2024) e em normas como a ISO/IEC/IEEE 24765, exige que três termos sejam usados sem ambiguidade:

- Erro (error, mistake): ação humana que produz um resultado incorreto. Ex.: um desenvolvedor interpreta mal um requisito.
- Defeito (defect, bug, fault): a manifestação desse erro no artefato de software (código, documento, modelo). Ex.: condição if (idade > 18) onde deveria ser >=.
- Falha (failure): desvio visível do comportamento esperado, observável quando o software é executado. Ex.: usuário de 18 anos não consegue se cadastrar.

Por que a distinção importa? Porque um defeito pode permanecer dormente no sistema e só manifestar-se em condições específicas. Se sua política de qualidade mede apenas "falhas", você está medindo o que o cliente percebeu; se mede "defeitos", está medindo o que o sistema contém. Métricas distintas, decisões distintas.

MINI-EXEMPLO

Um programador (erro) troca `<=` por `<` em um cálculo de desconto (defeito). O código pode ser executado milhares de vezes sem que a falha apareça — até o dia em que um cliente com total exatamente igual ao limite tente comprar. Nesse instante, a falha manifesta-se.

1.3 Verificação × Validação

Verificação e validação são dois lados da mesma moeda: garantir que o produto é adequado. A diferença está na pergunta que cada uma responde:

- Verificação (verification) — "Estamos construindo o produto corretamente?" Confere conformidade com a especificação. Pode ser feita SEM executar o código (revisões, inspeção, análise estática).
- Validação (validation) — "Estamos construindo o produto certo?" Confere se o software atende à real necessidade do usuário. Exige execução (testes dinâmicos, UAT, beta).

Um software pode ser perfeitamente verificado e mesmo assim falhar na validação — por exemplo, se o requisito original estava errado. Essa é a razão pela qual times modernos integram Product Owners, UX e QA desde o refinamento do backlog: é mais barato descobrir que o requisito estava errado antes do código existir.

1.4 Os Sete Princípios de Teste (ISTQB CTFL v4.0)

Estes princípios formam a base teórica universal do teste moderno. Eles são cobrados no CTFL, no ENADE e são aplicáveis a qualquer projeto, independentemente de metodologia.

1. Testes mostram a presença de defeitos, não a ausência.

Por mais que você teste, nunca poderá afirmar que o software está 100% livre de defeitos. Testar reduz a probabilidade de defeitos não descobertos, mas não prova ausência.

2. Teste exaustivo é impossível.

O número de combinações de entradas, estados e tempos é praticamente infinito, mesmo em sistemas simples. Use técnicas de teste + análise de risco para priorizar.

3. Testar cedo economiza tempo e dinheiro (shift-left).

Defeitos achados perto da origem custam dezenas a centenas de vezes menos para corrigir do que em produção. Envolve QA no refinamento de requisitos e code review.

4. Defeitos se agrupam (clustering).

Pareto se aplica: aproximadamente 20% dos módulos concentram 80% dos defeitos. Priorize testes nos módulos mais propensos — muitas vezes os mais complexos ou recentemente alterados.

5. Paradoxo do pesticida.

Os mesmos testes rodados repetidamente perdem eficácia, como pesticidas contra pragas que se adaptam. Revise e atualize regularmente a suíte de testes.

6. Teste depende do contexto.

Software de marca-passo tem exigências totalmente diferentes de um e-commerce ou de um chatbot. Não existe "o jeito certo" universal.

7. Ausência de erros é uma ilusão.

Um software pode estar livre de bugs e ainda assim ser inútil — se não resolve o problema real do usuário. Teste de usabilidade e validação de negócio são tão importantes quanto caixa-preta.

APLICAÇÃO PRÁTICA

Em uma retrospectiva de sprint, usar estes princípios como "pauta" força o time a conversar sobre qualidade sem cair em opinião — cada afirmação pode ser traçada a um princípio.

1.5 A ISTQB e a certificação CTFL v4.0.1

A ISTQB (International Software Testing Qualifications Board) foi fundada em 2002, em Edimburgo, Reino Unido. É uma entidade sem fins lucrativos, gerida por voluntários de Member Boards

espalhados por mais de 120 países. Sua missão é manter um vocabulário comum, um conjunto de syllabi e uma rede de exames para qualificação profissional em teste de software.

No Brasil a ISTQB é representada pelo BSTQB (Brazilian Software Testing Qualifications Board), que também oferece o exame em português. Em 2025 já foram emitidas mais de 1,2 milhão de certificações ISTQB em todo o mundo, e o syllabus CTFL v4.0 ultrapassou 746 mil downloads oficiais.

Trilha de certificações

A trilha ISTQB é organizada em três patamares (Foundation, Advanced, Expert) e trilhas laterais ("streams") de especialização:

- Foundation Level (CTFL v4.0.1) — pré-requisito para quase todas as demais. 64 objetivos de aprendizagem, 40 questões no exame, 60 minutos, 65% para aprovar.
- Advanced Level — Test Manager, Test Analyst, Technical Test Analyst.
- Expert Level — Improving the Test Process, Test Management.
- Streams — Agile Tester, Mobile Testing, Performance Tester, AI Testing (2024), Security Tester, Test Automation Engineer, Model-Based Tester, Usability Tester.

Por que importa para você

Pesquisas de remuneração em 2025 (Glassdoor, VagasQA) apontam que profissionais de QA com certificação CTFL ganham em média 18% mais que seus pares não-certificados no mesmo nível de experiência. Para um recém-formado do 6º período de ADS, o CTFL é um diferencial acessível (exame por volta de R\$ 850 pelo BSTQB) e vitalício (v4.0 não exige renovação).

DICA DO PROFESSOR

O conteúdo desta disciplina cobre cerca de 70% do syllabus CTFL v4.0.1. Complementando com 500+ questões de simulado (astqb.org e utest.com) e leitura do syllabus, é possível se preparar para o exame em 3-4 meses de estudo regular.

1.6 Processo de Teste e Níveis de Teste

1.6.1 Sete atividades do processo de teste (ISTQB v4)

Planejamento de Teste

Define objetivos, escopo, riscos e estratégia geral.

Monitoração e Controle

Mede progresso em relação ao plano e ajusta continuamente.

Análise

Identifica o que testar a partir das bases de teste (requisitos, código, riscos).

Projeto (Design)

Define COMO testar — casos de teste derivados por técnicas.

Implementação

Prepara ambientes, dados, scripts automatizados e material de apoio.

Execução

Roda os testes, registra defeitos, anota resultados.

Conclusão (Test Completion)

Relatórios, lições aprendidas, arquivamento de ativos.

AGILE ≠ SEM PROCESSO

Em times ágeis todas essas atividades ocorrem — só que DENTRO de cada sprint, não em fases sequenciais. A Monitoração e Controle (atividade 2) atravessa as outras seis.

1.6.2 Níveis de Teste e o Modelo em V

O modelo em V mostra a correspondência entre cada fase de desenvolvimento e um nível de teste. Embora originalmente associado ao ciclo cascata, o modelo continua útil para entender que cada artefato de desenvolvimento deve ter um teste correspondente:

Nível	Base de teste	Quem faz	Ferramentas típicas (2025)
Unitário	Projeto detalhado, código	Desenvolvedor	JUnit, PyTest, Jest, XUnit
Integração	Projeto de interfaces, arquitetura	Dev / QA	REST Assured, Postman, Testcontainers
Sistema	Análise de sistema, requisitos	QA	Selenium 4, Cypress, Playwright
Aceitação	Requisitos de negócio, critérios do cliente	PO / Cliente / QA	Cucumber, FitNesse, Gherkin

Hoje é comum complementar com shift-right: monitorar em produção por observabilidade, canary releases e feature flags. A ideia não substitui os níveis anteriores; acrescenta uma camada final.

1.7 Tipos de Teste

Enquanto níveis respondem ao "quando" testar (em qual fase), tipos respondem ao "o que" queremos verificar. O CTFL v4.0 consolida quatro grandes categorias:

Testes funcionais

Verificam O QUE o sistema faz. Cobrem regras de negócio, integridade de dados, entradas e saídas.

Testes não-funcionais (ISO/IEC 25010)

Verificam COMO o sistema se comporta sob certos aspectos:

- Performance / Eficiência — tempo de resposta, vazão (JMeter, k6, Gatling) •
- Segurança — OWASP Top 10 (ZAP, Burp, SonarQube)
- Usabilidade — heurísticas de Nielsen (Hotjar, testes A/B)
- Confiabilidade — MTBF, tolerância a falhas (Chaos Engineering)
- Portabilidade — multi-browser, multi-device (BrowserStack)
- Manutenibilidade — SonarQube, Code Climate, dívida técnica
- Compatibilidade — integração com APIs externas (Postman, Karate)

Testes de caixa-branca (estruturais)

Dirigidos pelo código. Avaliam quão bem a suíte cobre linhas, ramos, condições,

caminhos. **Testes relacionados a mudanças**

Duas categorias distintas, frequentemente confundidas:

- Reteste (confirmation testing) — re-executar o MESMO caso após a correção de um defeito, para confirmar que o fix funcionou.
- Regressão (regression testing) — executar OUTROS casos que antes estavam OK, para detectar efeitos colaterais introduzidos pela mudança.

PÍLULA CTFL 1.7

No exame CTFL é recorrente uma questão confundindo reteste e regressão. Decore: reteste = mesmo teste; regressão = outros testes que protegem o que funcionava.

Capítulo 2 — Técnicas de Teste

Uma técnica de teste é uma forma sistemática de escolher quais casos executar. Ela resolve dois problemas clássicos: (a) evita testar só o caminho feliz e (b) permite justificar a decisão de parar (critério de cobertura mensurável). O CTFL v4.0, capítulo 4, divide as técnicas em três famílias: caixa preta, caixa-branca e baseadas em experiência.

2.1 Testes estáticos e revisões

Testes estáticos analisam o artefato SEM executá-lo. Podem ser manuais (revisões) ou automatizados (análise estática).

2.1.1 Níveis de formalidade de revisão

- Informal — revisão "de ombro", sem registro formal. Ex.: pair programming.
- Walkthrough — autor apresenta, equipe discute. Foco em aprender sobre o artefato.
- Technical Review — reunião facilitada, com papéis definidos e checklist técnico.
- Inspeção (Fagan) — formal, com moderador, leitor, métricas de defeitos. Ideal para artefatos críticos.

2.1.2 Análise estática automatizada

Ferramentas como SonarQube, SpotBugs e PMD analisam o código-fonte sem executá-lo. Detectam: null reference, dead code, alta complexidade ciclomática, falhas de segurança (OWASP Top 10) e violação de padrões de codificação. Integradas ao pipeline, podem bloquear o merge quando a qualidade não atinge o limite configurado (Quality Gate — ver Capítulo 3).

2.2 Técnicas de caixa-preta (black-box)

2.2.1 Partição de Equivalência

Divide o domínio de entrada em classes que "sofrem o mesmo destino": todos os valores de uma classe produzem o mesmo comportamento. Testa-se um representante de cada classe, reduzindo drasticamente o número de casos.

Exemplo — validar idade para carteira de motorista ($18 \leq \text{idade} \leq 80$):

- Classe 1 — idade $< 18 \rightarrow$ INVÁLIDA (ex.: 10)
- Classe 2 — $18 \leq \text{idade} \leq 80 \rightarrow$ VÁLIDA (ex.: 35)
- Classe 3 — idade $> 80 \rightarrow$ INVÁLIDA (ex.: 95)

3 casos em vez de 100! Mesma confiança, muito menos esforço.

2.2.2 Análise de Valor Limite (BVA)

Complementa a partição testando os limites de cada classe — onde os bugs costumam se esconder (bugs off-by-one, erros em condicionais). Normalmente testa-se o valor no limite, um abaixo e um acima.

Exemplo — ainda com a regra $18 \leq \text{idade} \leq 80$:

- Limite inferior: 17 (rejeita), 18 (aceita), 19 (aceita)
- Limite superior: 79 (aceita), 80 (aceita), 81 (rejeita)

2.2.3 Tabela de Decisão

Ideal para regras de negócio com múltiplas condições que se combinam. As condições são listadas nas linhas e cada coluna representa uma regra (uma combinação de valores V/F). As ações correspondentes ficam abaixo.

Exemplo — desconto para clientes VIP com compra $> \text{R\$ } 500$:

Condições	R1	R2	R3	R4
Cliente VIP?	V	V	F	F
Compra $> \text{R\$ } 500$?	V	F	V	F
Ações				
Desconto 20%	X	-	-	-
Desconto 10%	-	X	X	-
Sem desconto	-	-	-	X

4 regras $\rightarrow$ 4 casos de teste. Nenhuma combinação relevante fica de fora.

2.2.4 Transição de Estados

Ideal para sistemas que têm ESTADOS — pedido em e-commerce (carrinho $\rightarrow$ pago $\rightarrow$ enviado $\rightarrow$ entregue), login (deslogado $\rightarrow$ autenticando $\rightarrow$ logado $\rightarrow$ bloqueado), sessão de jogo, etc. Modela-se o sistema como um diagrama de estados (UML state machine) e derivam-se casos cobrindo:

- Todos os estados (coverage 0-switch)
- Todas as transições válidas (1-switch)
- Transições inválidas — o que NÃO deveria acontecer

2.2.5 Teste por Caso de Uso

Baseia-se em diagramas UML de casos de uso. Cada caso de uso descreve ator, pré-condições, fluxo principal, fluxos alternativos, fluxos de exceção e pós-condições. A técnica deriva, no mínimo, um caso de teste para cada fluxo — é excelente para teste de aceitação.

2.3 Técnicas de caixa-branca (white-box)

Exigem conhecer o código. Medem o que a suíte cobre internamente.

2.3.1 Cobertura de Comandos (Statements)

Meta: executar cada linha pelo menos uma vez. Exemplo:

```
Java
int absoluto(int x) {
    int r; // L1
    if (x < 0) { // L2
        r = -x; // L3
    } else {
        r = x; // L4
    }
    return r; // L5
}
```

- Caso 1: $x = -5 \rightarrow$ cobre L1, L2, L3, L5
- Caso 2: $x = 5 \rightarrow$ cobre L1, L2, L4, L5

Juntos, 100% de comandos. Limitação: um if sem else atinge 100% com um único caso verdadeiro — pode esconder defeitos.

2.3.2 Cobertura de Decisões (Branches)

Mais forte que cobertura de comandos: exige que cada desvio (T e F) seja executado ao menos uma vez. Inclui 100% de comandos automaticamente. É geralmente a exigência mínima em software certificado.

2.3.3 Cobertura de Condições e MC/DC

Em decisões compostas (ex.: $\text{if } (a \ \&\& \ b \ || \ c)$), nem toda condição individual é testada apenas com T e F do "resultado". Para isso existem métricas mais rigorosas:

- Cobertura de Condição — cada condição atômica avaliada em T e F.
- Condição/Decisão (C/DC) — combina condições e decisões.
- MC/DC (Modified Condition/Decision Coverage) — cada condição deve afetar o resultado da decisão de forma INDEPENDENTE. Exigência da norma DO-178C (aviônica).

2.3.4 Teste de Caminho e Complexidade Ciclomática

A métrica $V(G)$ de McCabe indica o número mínimo de caminhos independentes necessários para cobrir todas as combinações de decisões de um método:

$$V(G) = E - N + 2$$

onde E = arestas e N = nós do grafo de fluxo de controle. V(G) também é igual a (nº de decisões + 1). Guias de engenharia recomendam $V(G) \leq 10$ por método — acima disso, refatore.

2.4 Técnicas baseadas em experiência

Usadas quando a especificação é fraca, incompleta ou inexistente, ou quando o prazo é curto. Também complementam as demais técnicas.

Error Guessing

"Chute educado" sobre onde o bug pode estar, baseado em histórico, bugs conhecidos e intuição do testador.

Teste Exploratório

Aprender, desenhar e executar em paralelo. NÃO é teste caótico: usa Session-Based Test Management (SBTM) com charters (missões curtas), sessions (60-90 min) e debriefs. Integrado a ferramentas como Xray, TestRail e Zephyr.

Checklist-Based e Attack-Based

Checklists estruturados de "coisas a verificar" (ex.: ISO 25010). Testes baseados em padrões de ataque conhecidos (SQL injection, XSS, CSRF) alinhados ao OWASP Top 10.

2.5 Teste de mutação

"Quem testa os testes?" Testes de mutação introduzem pequenas mudanças no código (mutantes) e verificam se a suíte detecta cada mutante. Se algum teste falha, o mutante está morto (ótimo). Se todos passam, o mutante está vivo — evidência de teste cego.

Ferramentas: PIT (Java), MutMut (Python), Stryker.NET (.NET), Stryker (JS/TS). Meta realista: 70-85% de mutation score.

2.6 Qual técnica aplicar em cada contexto

Situação	Técnica recomendada
Campos com faixas válidas (idade, preço, CEP)	Partição de Equivalência + BVA
Regras de negócio com múltiplas condições	Tabela de Decisão
Sistema com estados (pedido, login, sessão)	Transição de Estados
Fluxo ponta-a-ponta de negócio	Teste por Caso de Uso
Validar cobertura do próprio teste	Caixa-Branca (decisão, MC/DC)
Spec ruim ou inexistente, prazo apertado	Exploratório + Error Guessing
Validar qualidade da suíte	Teste de Mutação

Capítulo 3 — Automação, Ferramentas e Qualidade

Este capítulo reúne o conhecimento prático da Etapa 3: como levar os conceitos e técnicas dos Capítulos 1 e 2 para dentro de um pipeline automatizado, medir qualidade e usar as principais ferramentas do mercado em 2026.

3.1 Pirâmide de Testes e ROI de Automação

Mike Cohn (2009) popularizou a pirâmide de testes como uma heurística para distribuir esforço entre níveis. A ideia central é: muitos testes unitários (rápidos, baratos, estáveis), alguns de integração e poucos end-to-end/UI (lentos, frágeis, caros). Variações modernas como o "Testing Trophy" (Kent C. Dodds) enfatizam testes de integração como base, especialmente para aplicações web.

Quando automatizar?

O retorno sobre o investimento em automação depende do equilíbrio entre custo de criar/manter e frequência de execução.

- AUTOMATIZE: regressão, contratos de API, smoke tests, validação de dados, performance.
- NÃO AUTOMATIZE: UX/visual, testes "únicos", exploratório, usabilidade, spec altamente volátil.

REGRA PRÁTICA

Se um teste vai rodar menos de 5 vezes ao longo do projeto, o tempo investido na automação provavelmente não se paga. Em contrapartida, um teste de regressão que roda a cada commit se paga em poucas semanas.

3.2 JUnit 5 e JUnit 6

JUnit é o framework mais usado para testes unitários em Java. A versão JUnit 5 (Jupiter) é um redesign completo sobre a anterior, lançado em 2017. Em setembro de 2025 foi lançado o JUnit 6, que mantém a mesma arquitetura do 5 mas eleva o baseline para Java 17+ e adota o sistema de módulos JPMS. A última versão estável da série 5 é 5.14.2 (junho/2025). Para projetos novos em 2026, recomenda-se começar já com Jupiter — 5.x LTS se ainda precisar compatibilizar com Java 11 ou 6.x para aproveitar modularidade.

3.2.1 Arquitetura modular

JUnit 5 é dividido em três módulos:

- JUnit Platform — executa testes na JVM; integra com IDE, Maven, Gradle, CLI.
- JUnit Jupiter — API de escrita de testes com `@Test`, `@Nested`, lambdas, extensões.
- JUnit Vintage — motor que roda testes antigos (JUnit 3/4).

Ambos os motores "plugam" na Platform via SPI TestEngine. Isso permite que Spock (Groovy), Kotest (Kotlin) e TestNG também rodem sobre a mesma plataforma.

3.2.2 Anotações essenciais

- `@Test` — marca um método como caso de teste.
- `@BeforeEach` / `@AfterEach` — setup/teardown antes/depois de cada teste.
- `@BeforeAll` / `@AfterAll` — executa 1x antes/depois da classe inteira (método deve ser static).
- `@DisplayName` — nome legível para o relatório.

- `@Disabled("motivo")` — pula o teste.
- `@Nested` — agrupa testes em classes internas.
- `@ParameterizedTest` — mesma lógica com múltiplos argumentos.
- `@RepeatedTest(N)` — roda o método N vezes.

3.2.3 Exemplo completo com padrão AAA

O padrão Arrange-Act-Assert (AAA) estrutura cada teste em três seções claras:

```
Java
import org.junit.jupiter.api.*;
import static

org.junit.jupiter.api.Assertions.*; class

CalculadoraTest {

    private Calculadora calc;

    @BeforeEach
    void setUp() {
        calc = new Calculadora(); // Arrange
    }

    @Test
    @DisplayName("Soma 2 + 3 deve retornar 5")
    void somaDoisNumerosPositivos() {
        int r = calc.somar(2, 3); // Act
        assertEquals(5, r); // Assert
    }

    @ParameterizedTest
    @CsvSource({ "2,3,5", "-1,1,0", "0,0,0" })
    void somaParametrizada(int a, int b, int esperado) {
        assertEquals(esperado, calc.somar(a, b));
    }
}
```

3.2.4 Asserções avançadas e expectativa de exceção

```
Java
// assertAll roda TODAS as asserções e reporta todas as
falhas @Test
void varias_asserções_em_um_teste() {
    Pessoa p = new Pessoa("Ana", 25);
    assertAll("Pessoa",
        () -> assertEquals("Ana", p.getNome()),
        () -> assertEquals(25, p.getIdade()),
        () -> assertTrue(p.isMaiorDeIdade())
    );
}
// assertThrows captura exceção esperada
```

```

@Test
void dividir_por_zero_lanca_excecao() {
    ArithmeticException e = assertThrows(
        ArithmeticException.class,
        () -> calc.dividir(10, 0)
    );
    assertEquals("/ by zero", e.getMessage());
}

```

3.3 Mockito e Test Doubles

Testes unitários precisam de ISOLAMENTO. Se o método que queremos testar depende de um DAO, um serviço externo ou do relógio do sistema, substituímos essas dependências por test doubles (dublês). Gerard Meszaros (xUnit Test Patterns, 2007) catalogou cinco tipos:

- Dummy — objeto só para preencher parâmetro, nunca é usado.
- Stub — retorna respostas pré-programadas para chamadas específicas.
- Fake — implementação simplificada e funcional (ex.: repositório em memória).
- Spy — grava chamadas para verificação posterior. Mock parcial.
- Mock — objeto com expectativas explícitas; falha o teste se não cumpridas.

Em Java, Mockito é o padrão de facto. Exemplo com JUnit 5 + Mockito 5:

```

Java
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.InjectMocks;
import org.mockito.MockitoAnnotations;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class PedidoServiceTest {

    @Mock PagamentoGateway gateway;
    @Mock EstoqueRepository estoque;
    @InjectMocks PedidoService service;

    @BeforeEach
    void init() { MockitoAnnotations.openMocks(this); }

    @Test
    void deveFinalizarPedidoComEstoque() {
        when(estoque.temDisponivel("SKU1")).thenReturn(true);
        when(gateway.cobrar(100.0)).thenReturn("tx-123");

        String tx = service.finalizar("SKU1", 100.0);

        assertEquals("tx-123", tx);
        verify(estoque).debitar("SKU1");
    }
}

```



3.4 TDD e BDD

3.4.1 TDD — Test Driven Development

Criado por Kent Beck (2003), o TDD propõe um ciclo curto de três passos: Red (escrever um teste que falha), Green (implementar o mínimo para passar) e Refactor (melhorar o código sem mudar o comportamento).

Em 2026, ainda que poucos times sigam TDD de forma estrita, a prática de escrever pelo menos um teste junto com o código entrega grande parte do benefício. Com assistentes de IA (Copilot, Claude), o test-first prompting se tornou um padrão para reduzir alucinações e validar automaticamente a sugestão.

3.4.2 BDD — Behavior Driven Development

BDD é um refinamento do TDD focado na linguagem de negócio. Cenários são escritos em Gherkin (Given/When/Then) e ligados ao código via step definitions. Exemplo:

Gherkin

Feature: Desconto VIP

Scenario: Cliente VIP com compra > R\$ 500

Given um cliente com status "VIP"

And o valor da compra é 600

When calcular o desconto

Then o desconto deve ser 20 por cento

Ferramentas: Cucumber (Java/JS/.NET), SpecFlow (.NET), Behave (Python), Karate (API). Integram-se com JUnit 5.

3.5 JaCoCo e Cobertura de Código

JaCoCo (Java Code Coverage) é a ferramenta padrão para medir cobertura em projetos Java. Instrumenta o bytecode em tempo de execução e gera relatórios em HTML/XML. Integra-se com Maven (jacoco-maven-plugin), Gradle (JaCoCo Plugin), SonarQube e pipelines de CI.

Métricas medidas

- Instructions — instruções de bytecode cobertas (mais fina que linhas).
- Branches — ramos de decisões.
- Lines — linhas de código-fonte.
- Complexity — complexidade ciclomática coberta.
- Methods — métodos executados.
- Classes — classes carregadas.

Metas realistas para projeto novo: $\geq 80\%$ de Instructions e $\geq 70\%$ de Branches. Perseguir 100% cegamente leva a testes triviais apenas para "fechar a conta" — use teste de mutação para validar a QUALIDADE dos testes.

3.6 SonarQube 2025 e Clean as You Code

SonarQube é a plataforma mais adotada para análise estática contínua. Em 2025 a linha Server tem duas versões de referência — 2025.1 LTA e 2025.4+ — e expandiu seu portfólio:

- SonarQube Server — self-hosted (Community gratuita; Developer e Enterprise pagas). • SonarQube Cloud — SaaS (antigo SonarCloud), integra direto com GitHub/Bitbucket/Azure DevOps.
- SonarQube IDE — antigo SonarLint: feedback em tempo real no VSCode/IntelliJ/Eclipse enquanto o desenvolvedor digita.

Novidades de 2025

- Detecção automática de código gerado por GitHub Copilot — aumenta a consciência sobre ownership e segurança.
- Cobertura completa MISRA C++ 2023 — relevante para sistemas embarcados.
- Melhorias de performance nos analisadores de Java, Python, JS/TS.
- Suporte a STIG para T-SQL — conformidade militar americana.
- Quality Gates mais flexíveis para administradores, com opção de gate não estritamente CAYC.

Clean as You Code (CAYC)

É a filosofia-chave do SonarQube: em vez de tentar limpar todo o código legado (normalmente inviável), foque apenas em código NOVO. O Quality Gate roda em cada pull request analisando apenas o diff. Com o tempo, naturalmente o codebase inteiro fica limpo — sem esforços épicos de "refactor-all-the-things".

Quality Gate padrão (Sonar Way, 2025)

Condição	Threshold
Coverage on new code	$\geq 80\%$
Duplicated lines on new code	$\leq 3\%$
Maintainability Rating on new code	A
Reliability Rating on new code	A
Security Rating on new code	A
Security Hotspots reviewed	100%

3.7 CI/CD, Quality Gates e métricas DORA

Integração contínua (CI) e entrega contínua (CD) são o contexto onde testes automatizados ganham poder. Um pipeline típico em GitHub Actions / GitLab CI / Jenkins executa:

- PUSH — commit dispara o job.
- BUILD — Maven/Gradle compilam.
- UNIT — JUnit 5 roda testes unitários em paralelo.
- COVERAGE — JaCoCo gera relatório.
- SONAR — análise estática + Quality Gate.
- INT/E2E — Testcontainers, REST Assured, Playwright.
- DEPLOY — staging automático, produção com aprovação manual ou canary.

Regras de ouro

- Fail fast — testes caros rodam depois dos baratos.
- Feedback em ≤ 10 minutos para devs — senão ninguém olha o resultado.
- Separar deploy de release com feature flags (LaunchDarkly, Unleash).
- Observabilidade pós-deploy (logs, métricas, traces) é TESTE em produção.

Métricas DORA (DevOps Research and Assessment, 2024-25)

- Deployment Frequency — quantas vezes/dia você entrega.
- Lead Time for Changes — tempo entre commit e produção.
- Change Failure Rate — % de deploys que causam incidente.
- Mean Time To Restore — tempo médio de recuperação após incidente.

Times "Elite" (relatório DORA 2024) fazem múltiplos deploys por dia, têm lead time de menos de 1 hora e MTTR de menos de 1 hora. O oposto (times "Low") fazem deploy mensal ou raro, com lead time de meses.

3.8 Panorama 2026 de ferramentas

Camada Unitário	Ferramentas principais (2026) JUnit 5/6 (Java), PyTest (Python), Jest/Vitest (JS/TS), NUnit, XUnit.net
Mocks	Mockito 5, EasyMock, unittest.mock, Sinon.js, MockK (Kotlin)
API / Integração	REST Assured, Postman/Newman, Karate, Testcontainers
E2E / UI	Playwright (líder 2025), Cypress, Selenium 4, Puppeteer
Performance	k6, JMeter, Gatling, Locust
Qualidade / SAST	SonarQube, SpotBugs, PMD, CodeQL, Snyk

TENDÊNCIA 2026

Playwright tem dominado o espaço de E2E web por sua abstração cross-browser, auto-wait e paralelismo. Cypress ainda é forte em equipes JS/TS com necessidade de DX simples.

Referências

ISTQB. Certified Tester Foundation Level Syllabus v4.0.1. International Software Testing Qualifications Board, 2024. Disponível em: [istqb.org](https://www.istqb.org).

ISTQB. Glossary of Software Testing Terms v4.5. International Software Testing Qualifications Board, 2024.

PEZZÈ, M.; YOUNG, M. Teste e Análise de Software: Processos, Princípios e Técnicas. Bookman, 2008.

BECK, K. Test Driven Development: By Example. Addison-Wesley, 2003.

MESZAROS, G. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley, 2007.

COHN, M. Succeeding with Agile. Addison-Wesley, 2009 (Test Pyramid).

SONAR. SonarQube Server 2025.4 Documentation. Disponível em: docs.sonarsource.com.

JUNIT TEAM. JUnit 5 User Guide (versões 5.14.x) e JUnit 6 Release Notes. Disponível em: docs.junit.org.

OWASP Foundation. OWASP Top 10 - 2021/2024. Disponível em: owasp.org.

DORA / Google Cloud. State of DevOps Report 2024. Disponível em: dora.dev. ISO/IEC 25010:2023. Systems and software Quality Requirements and Evaluation (SQuaRE).

Material didático elaborado por Prof. Dr. Diogo Vinícius de Sousa Silva, CTFL. Distribuição autorizada para alunos do IFMA — Campus Coelho Neto. Abril/2026.